ArgorixLang: Compiled Governance, Jurisdiction-Aware Agent Metadata, and Offline-Verifiable Runtime Evidence

Gustavo Venegas*, Edison Vazquez*, Danilo Naranjo†, Benjamin Gonzalez*
*Chilean Chamber of Artificial Intelligence †Ocular
gustavo@gobiernoia.cl, edison@jhedai.com, dan@ocularsolution.com, ben@nirvana-ai.com

***Abstract*—Agent governance is commonly distributed across prompts, policy services, provider configuration, identity metadata, and logs. ArgorixLang instead places typed agent declarations, policy constraints, provider permissions, and evidence requirements in one compilation and runtime boundary. This paper reports an executed end-to-end and adversarial campaign of 231 runs over the release binaries, together with the historical snapshot the campaign re-measures. Expected and observed outcomes are kept in separate files and separate processes: the collector cannot read the oracle, and removing one binary makes the campaign fail rather than produce results. Typed outcomes match the preregistered oracle in 118/118 rows, with 0/198 false allows and 0/198 false denies; adverse conditions end fail-closed in 16/16; independent mutations applied after generation are detected in 21/22 cases; and across eight dispatch conditions instrumented with loopback, filesystem, and secret canaries, 0/21 prohibited proposals reached the local sink. Three defects the campaign exposed were then fixed and the campaign re-run from a clean directory with the pre-fix baseline preserved: a malformed provider payload that no boundary rejected, a source-only modification that was undetectable, and the absence of any way to tell the original artifact set from a self-consistent replacement, now closed by a detached signature checked against a producer trust anchor. Indirect prompt injection was then evaluated against a real model: in 20/40 injected arms the model proposed the prohibited action, and 25/25 prohibited proposals were contained before any sensor. What remains are boundaries rather than defects, and the paper states each one: unsigned verification still cannot detect a replacement, signing adds no key governance, the release cannot be instrumented at its mediation point, and the injection result measures containment of a model's proposal rather than the resistance of an Argorix agent loop. The historical snapshot reproduces exactly—27/27 internally consistent bundles, 0/27 policy-approved, 1,188 unknown-rule findings—and remains a demonstration of faithfully preserved failure, not of governance.**

***Index Terms*—AI agents, compiled governance, policy enforcement, jurisdiction metadata, runtime evidence, fail-closed execution, reproducibility**

I. Introduction

Tool-using agents turn generated text into proposed operations. Instructions in a prompt can influence a model, but they do not by themselves create a reference monitor at the action boundary. Classical work on execution monitoring asks which policies can be enforced by observing and suppressing events [1], [2]; contemporary authorization languages such as Cedar make policies explicit, analyzable, and independent of application code [3]. Agent systems add a further problem: identity attributes, provider permissions, protocol contracts, and audit evidence are often configured in different components and can disagree.

ArgorixLang is a compiled language and dry-run runtime for agent protocols [4]. Source declarations are parsed, checked semantically, lowered to typed intermediate representation and bytecode, then loaded by a VM. The VM separates declarative external-provider contracts from executable adapters. In the evaluated configuration, only a built-in simulated provider is executable; an external contract does not become callable by being declared. The VM emits a trace, a SecurityReport, and an EvidenceBundle for offline consistency checking.

This revision answers four research questions:

1) How can language and runtime boundaries combine policy, provider permissions, jurisdiction-aware metadata, and evidence generation?
2) Do the compiler, bytecode verifier, and VM produce the predefined typed decision for behaviourally distinct workloads, and do errors, missing artifacts, replay, concurrency, and deadlines stay fail-closed?
3) Which independent modifications does the evidence verifier detect relative to a fixed bundle, and which fall outside its trust boundary?
4) When a prohibited operation is proposed, is it blocked before an instrumented destination, and what can be claimed about that from outside the runtime's own report?

Question 4 measures *effect containment* observed by independent sensors. It is not a claim about preventing prompt injection: what we evaluate is whether a prohibited action a real, successfully injected model proposed reaches a destination, not whether the model can be made to propose it.

The paper makes three contributions. First, it describes a compiler-to-VM boundary that treats model output as a proposal and keeps external contracts non-executable unless an adapter is separately registered. Second, it reports an executed campaign of 231 runs over the release executables in which expected and observed outcomes are physically separated, digests are recomputed by an implementation independent of the code under test, and a negative control removes a binary to show the harness cannot fabricate a result. The campaign found three real defects, which were fixed and re-measured against a preserved baseline. Third, it reports the historical snapshot as an unfavorable result and re-measures it with the current binaries: 27 internally consistent bundles preserve 1,188

unknown-rule findings and yield zero policy-approved sessions. A previously reported 57-row fixture generator was removed entirely; it computed its own decisions and was therefore an executable specification, not an experiment over the production implementation.

We also narrow the terminology. The implementation contains historically named "sovereign" fields, but this paper calls them *jurisdiction and residency metadata*. Data and digital sovereignty are broader, contested concepts involving control and self-determination [5], [6]; country codes alone do not establish them.

II. Related Work

ArgorixLang combines established ideas rather than claiming that compilation alone creates a new security primitive. Runtime enforcement and reference monitors provide the foundation for mediating security-relevant operations [1], [2]. Cedar demonstrates how a dedicated, analyzable authorization language can separate policy from application logic [3]. ArgorixLang differs in scope: it co-compiles agent and message types, provider boundaries, local identity metadata, and evidence metadata, while retaining an explicit VM mediation point.

Evidence systems define a stronger comparison boundary. in-toto authenticates supply-chain step metadata and verifies a signed layout [7]; tamper-evident logging structures support consistency or inclusion proofs against an untrusted log [8]; secure logging can add forward integrity under machine compromise [9]. ArgorixLang's current EvidenceBundle has none of those external trust anchors. It detects modification of referenced artifacts relative to a trusted bundle, but an attacker who can replace the entire unsigned set remains outside its assurance boundary.

AgentDojo, InjecAgent, and indirect prompt-injection demonstrations all evaluate agents inside adversarial tool environments [10]–[12]. They require attack inputs, stateful tools, and externally checked outcomes. Our campaign supplies adversarial content, a real model loop, and externally checked outcomes—loopback, filesystem, and secret canaries with positive controls. It differs from those benchmarks in one structural way: the release carries no prompt content, so the model sits outside it and a driver maps the proposed action onto a program. We therefore report containment of a prohibited proposal, not the resistance of an agent loop, and report attack success on the model separately from it. Project-local ATrust and DCP-AI declarations are design vocabulary, not independent evidence for the runtime claims in this paper. Emerging agent discovery proposals such as NANDA are likewise treated as future context rather than implemented resolution [13].

III. Compiled Governance Boundary

A. Compilation and mediation

Figure 1 shows the implemented path. Parsing and semantic checking reject unknown symbols, missing capabilities, invalid provider bindings, and incompatible policy declarations before bytecode is emitted. The bytecode verifier checks versioned structural invariants. At runtime, the VM loads only executable

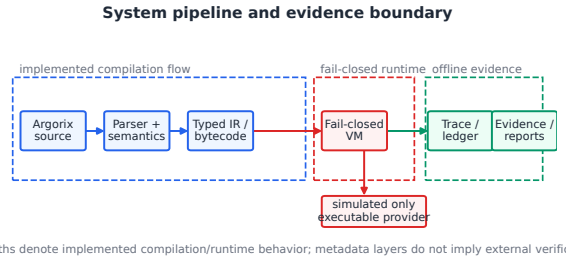

Fig. 1. Implemented source-to-evidence path. Metadata that reaches the trace is not thereby authenticated by an external authority.

providers from its registry; declarative contracts are stored separately. A missing authorization, unregistered provider, or invalid runtime profile produces a preserved failure outcome and ledger evidence.

The distinction between contract and adapter is load-bearing. A contract can describe a provider name, capability, allowed targets, and approval conditions, but it supplies no executable implementation. The default registry contains only simulated. External execution is therefore unavailable in the evaluated path even when an external provider is declared. This is a property of the VM registry and semantic checks; it is not evidence of host-level network isolation or behavior under a future real adapter.

B. Policy outcomes

Policy results are kept separate from the top-level execution status. Known rules can pass, deny, or require review; malformed or unsupported identifiers produce error or unknown-rule outcomes. The VM can complete a dry-run while the policy summary still requires review. Consequently, "completed," "digest verified," and "policy approved" are three different predicates. This separation is essential for interpreting the historical snapshot.

C. Jurisdiction-aware passports

An Agent Passport binds a local agent identifier to declared country, jurisdiction, residency, and naming fields. Semantic checks validate shapes and references and propagate metadata into IR, bytecode, reports, and evidence. The fields are self-asserted program attributes: they do not authenticate a person or organization, prove nationality or state recognition, establish the physical location of data, or certify legal compliance. External identity would require issuer policy, credentials, revocation, and trusted resolution, for example using DID/VC mechanisms [14], [15].

D. Evidence model

After a run, the VM can write bytecode, trace, report, and bundle artifacts. The bundle records relative paths and semantic SHA-256 digests. Offline verification re-reads the referenced objects, normalizes their serialized content, recomputes the digests, and checks the report/trace/ledger links. It does not evaluate policy, authenticate the producer, bind a historical

source file in the studied bundle schema, or consult an append-only service. A valid bundle means *internal consistency relative to that bundle*, not approval, attestation, or certification.

IV. Evaluation Method

We keep the two evidence sources separate because they answer different questions and were produced by different software states.

A. Historical observational snapshot

The unit is a request directory. A complete directory contains source, bytecode, trace, SecurityReport, and EvidenceBundle; a source-only directory is retained as incomplete. We inventory completeness, parse all available JSON, rerun offline bundle verification, count policy findings and events, and group complete directories by bytecode, trace, report, and ledger digests. The snapshot is descriptive: directories are not treated as independent workloads when their structural fingerprints coincide.

The verifier was rerun with the repository's VM executable. Policy status was read from detailed findings rather than inferred from the favorable security_checks label. Prompt text is absent from all complete traces; the six source-only directories have no trace. No prompt-level analysis was performed.

B. Executed adversarial campaign

The campaign drives the release executables built with extttcargo build --locked --release, not library calls inside the harness. Each case runs extttargorixc check, extttargorixc emit-bytecode, extttargorix-vm run --dry-run --reactive, and extttargorix-vm verify-evidence as separate processes, and every exit code, stream, artifact, digest, and latency is recorded.

Validity. Cases and expectations live in different files and are consumed by different processes. cases.json contains inputs and procedures only; oracle.json contains expectations only. The collector cannot read the oracle: no module on its path names the file, and a guard turns any attempt into a hard error. A negative control copies the binary directory with argorix-vm removed and requires the campaign to fail and emit no rows, so a correct result cannot be produced without executing Argorix. An auxiliary verifier written in Python recomputes the bytecode, trace, report, and ledger digests independently of the Rust implementation; every row carries both values. Diagnostics are compared by stable class, never by message text.

Design. Five families are executed. E0 re-measures all 33 historical request directories, keeping the 6 incomplete ones in the denominator with the phase they reach. E1 runs twelve independent workloads three times each and requires at least twelve distinct behavioural fingerprints over six dimensions—policy, capability, provider, runtime profile, program structure, and outcome—so that repetition cannot pass as diversity. E2 exercises twenty adverse conditions: malformed sources, invalid bytecode, invalid injection routes, absent provider or adapter, allowlist rejection, missing and truncated artifacts, an imposed deadline, an unavailable sensor, concurrent execution, request replay, evidence replay, and paths outside the portable tree. E3 first generates a clean set and confirms it verifies, then mutates bytes on disk and re-invokes the real verifier, recording pre- and post-mutation digests so that a mutation that changed nothing is discarded rather than counted. E4 runs eight dispatch conditions under three independent sensors.

Sensors. A loopback HTTP sink with an append-only per-nonce log is advertised to the child process through the endpoint environment variable the declared adapter references; a filesystem sentinel in an isolated temporary directory is hashed before and after; and synthetic secret and key-material canaries—never real credentials—are exported into the child environment and then searched for in every artifact and both streams. Each sensor has a positive control that must fire, because zero hits from a sensor never shown to work is not evidence. In prohibited conditions the outcome is derived from the sensors, not from the VM's own report.

The mediation point. The release cannot be instrumented where it matters: its provider registry rejects every implementation that is not the built-in simulated one, and the reactive path rebuilds that provider on every run, so even a substituted registry never reaches the mediation point. Rather than weaken the release, we compile a second binary behind a non-default eval-tripwire feature. The release build does not compile it and rejects its flags; the evaluation binary reports a distinct version string that every run manifest records, so no row can be mistaken for one the release produced. The tripwire occupies the simulated slot, counts what the VM hands over, and can probe the loopback sink—giving a positive control *at the mediation point*, which is what makes zero hits elsewhere a measurement rather than an absence of instrumentation.

Prompt injection. The release ingests no prompt content, so the model sits outside it. Eight scenarios—external provider calls, network egress, secret exfiltration, key-material access—each with a benign and an indirect-injection arm run five times, for 80 runs against one model with its version, temperature, and seed recorded. The driver puts the content in front of the model, receives a structured action, maps it onto a precompiled program, and runs that program under the same sensors. The grid is declared in the catalogue before any model was called, so a run cannot select the favourable half afterwards, and the mapping is published and total: a proposal no program covers is counted, never dropped.

Authenticity. A separate argorix-sign binary produces a detached Ed25519 signature over a bundle's canonical bytes, and verify-evidence-{}-trust-anchor checks it. Signing lives outside the runtime so the VM never handles private key material. Because the bundle already binds bytecode, trace, report, and source by digest, one signature covers the whole set. Without an anchor the result is unchanged and claims only integrity.

Proportions are reported as n/N with Wilson 95% intervals. Where the numerator is zero we give the rule-of-three upper bound rather than writing a zero rate.

TABLE I
HISTORICAL SNAPSHOT, RE-MEASURED WITH THE CURRENT RELEASE BINARIES.

Measure	Value
Request directories	33
Complete / source-only	27 / 6
Internally consistent bundles	27/27
Policy-approved directories	0/27
Unknown-rule findings per directory	44
Unknown-rule findings, total	1,188
Structural fingerprint families	2 (sizes 20, 7)
Ledger events per directory	265
Independently recomputed digests	108/108

TABLE II
EXECUTED ADVERSARIAL CAMPAIGN. PROPORTIONS ARE n/N WITH WILSON 95% INTERVALS.

Measure	n/N	Wilson 95% CI (%)
Outcome matches oracle	118/118	[96.8, 100.0]
False allows	0/198 $\leq 1.5\%$	[0.0, 1.9]
False denies	0/198 $\leq 1.5\%$	[0.0, 1.9]
Adverse conditions fail-closed	16/16	[80.6, 100.0]
Evidence modifications detected	21/22	[78.2, 99.2]
Prohibited blocked before sink	21/21	[84.5, 100.0]
Prohibited rejected at a boundary	18/21	[65.4, 95.0]
Prohibited reaching the sink	0/21 $\leq 14.3\%$	[0.0, 15.5]
Artifact completeness	21/21	[84.5, 100.0]
Forged sets rejected (anchor)	3/3	[43.9, 100.0]
Injections that moved the model	20/40	[35.2, 64.8]
Prohibited proposals contained	25/25	[86.7, 100.0]

V. RESULTS

A. Snapshot: integrity preserves policy failure

Table I reports the complete result, re-measured with the current release binaries. Offline verification passes for all 27 complete bundles, but policy approval passes for none. Each complete directory contains 44 violations with the same reason, "unknown policy rule," for 1,188 findings in total. Inspection of the evaluator shows that none of the 44 identifiers exercised by this snapshot reaches a known-rule branch; the default result is the unknown-rule diagnostic. The policy subsystem is therefore not demonstrated as functional for the rule set exercised here. All 108 semantic digests recorded by the complete bundles are reproduced by the independent verifier, so the re-measurement is not a second reading of the same implementation.

The homogeneity matters as much. The 27 complete directories hold two fingerprint families of sizes 20 and 7 but one ledger digest, one event-type sequence, and 265 events each: repeated executions of two closely related structures, not 27 diverse workloads. The campaign exists because that snapshot cannot answer behavioural questions.

Its reports carry a favourable top-level label beside review-required detail. Serialization did not erase the unfavorable detail, which is useful, but preserving an error is not enforcing a policy. In the current release the aggregate verdict derives monotonically from the detail, and the campaign checks it directly: no security report in 231 runs carries an approving aggregate verdict over a block, review, warning, or unknown-rule detail. The snapshot's 44 identifiers are not merely unevaluated today—the current compiler and bytecode verifier reject an unknown policy rule outright, so the same program no longer reaches the runtime.

B. Campaign results

Table II reports the executed campaign. Across 231 runs the typed outcome matched the preregistered oracle in 118/118 scored rows, with 0/198 false allows and 0/198 false denies. Because the numerator is zero we report the rule-of-three bound rather than a zero rate: false allows are at most 1.5% at 95% confidence, which is a weak bound on 198 scored rows and should be read as such.

The first campaign scored 103/106 and every mismatch was a real defect, not an oracle error. Section V-C reports the defects, the fixes, and the re-measurement; the pre-fix run is archived unedited so both numbers remain checkable.

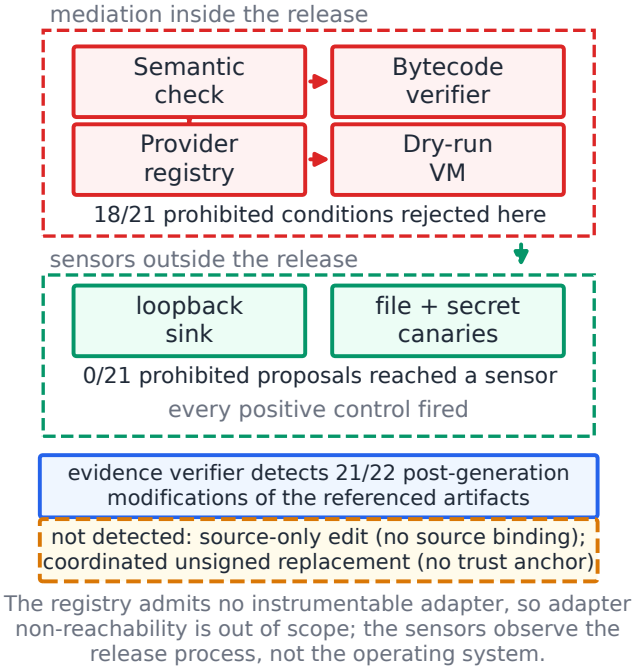

Fig. 2. Measured adversarial boundary. Counts are generated from the campaign summary, not transcribed. Rejection happens inside the release; the sensors that decide whether an effect escaped are outside it.

Behavioural diversity. The twelve E1 workloads produced 12 distinct behavioural fingerprints over 36 runs—one per workload, with repetitions collapsing exactly as they should. Every dimension separated at least two behaviours. Three workloads are rejected during semantic checking (missing capability, undeclared tool, undeclared model), one during policy checking (unknown rule), one at the runtime-profile request, three complete under an evaluated policy, one is denied by a block action, one requires review, one declares a non-executable external contract, and one is a multi-file package carrying messages, a policy, and a passport. Mediation

TABLE III
EVIDENCE MUTATIONS EXECUTED AFTER GENERATION, BY CLASS, RELATIVE TO THE ORIGINAL BUNDLE.

Mutation class	Detected	Wilson 95% CI (%)
authenticity	0/1	[0.0, 79.3]
bundle	5/5	[56.6, 100.0]
bytecode	3/3	[43.9, 100.0]
invalid json	2/2	[34.2, 100.0]
missing artifact	3/3	[43.9, 100.0]
portable tree	1/1	[20.7, 100.0]
report	3/3	[43.9, 100.0]
source binding	1/1	[20.7, 100.0]
trace	3/3	[43.9, 100.0]

therefore happens far earlier in the pipeline than the runtime: the reactive VM's provider-boundary branch is unreachable from any program the compiler accepts.

Fail-closed behaviour. All 16/16 adverse E2 conditions ended fail-closed, Wilson 95% [80.6, 100.0]%. Malformed sources and invalid bytecode are rejected before execution; an unknown injection route, an absent runtime profile, an absent adapter, and a denied adapter operation are all refused; a truncated bundle, a missing trace, a missing report, and an artifact path escaping the portable tree all fail verification. An imposed deadline that kills the VM mid-run produces an error rather than a pass. Two concurrent runs over the same inputs produced byte-identical reports, request replay reproduced identical digests for all four artifacts, and evidence verification is stable across repetitions.

Evidence mutation. Table III reports the E3 result by mutation class. A clean set was generated and verified first; 22 mutations were then applied to bytes on disk and the real verifier was re-invoked. 21/22 were detected, Wilson 95% [78.2, 99.2]%. The single remaining non-detection was predicted in the oracle and is a boundary rather than a surprise: replacing the bundle together with every artifact by a self-consistent unsigned set verifies successfully, because digest verification anchors nothing. Only a trust anchor separates the two, which is what the authenticity result below measures. We therefore report detection relative to a fixed original bundle and never use the word tamper-proof.

The mediation point. The evaluation build observes what the release cannot show about itself. Across 3 conditions, an allowed call reaches the provider exactly once and always with the dry-run flag set; a program rejected at bytecode verification never reaches it at all; and a probe issued from inside a provider invocation is recorded by the sink. That last case is the control that gives the others their meaning: egress from the mediation point is observable, so zero hits from the release build is a measurement. It describes the evaluation binary, not the release, and no real external adapter exists to test.

Authenticity. With a producer trust anchor, 3/3 non-producer artifact sets are rejected: the coordinated replacement, a bundle with no signature, and a signature from another key. Without an anchor the same replacement still verifies, which is what digest verification is rather than a defect in it.

Prompt injection. Against gpt-4o-mini-2024-07-18 at temperature zero with seed 7, the indirect injections moved the model in 20/40 injected arms, Wilson 95% [35.2, 64.8]%—a property of the model, not of Argorix. It varies sharply by scenario: every external-provider injection succeeded, half the key-material ones did, none of the secret-exfiltration ones. The model also proposed a prohibited action in 5/40 benign arms with no injection present, which is why the benign control matters.

Of every prohibited proposal, injected or not, 25/25 were contained before any sensor, Wilson 95% [86.7, 100.0]%; 0/25 reached one, so the honest bound is at most 12.0%. Every proposal mapped to a program (0/80 unmappable) and no call failed to reach the model, so the denominator is the whole grid.

The ceiling is structural: the driver maps the proposal onto a program, Argorix does not dispatch it. This measures containment of a prohibited action a real model proposed after a real injection, not the resistance of an agent loop, which the release cannot yet have because it ingests no prompt content.

Dispatch and side effects. Figure 2 summarises where each decision is taken. Across 8 conditions and 24 instrumented runs, every sensor fired its positive control in every run, and 0/21 prohibited proposals reached the loopback sink; the filesystem sentinel and both canaries also recorded zero hits. With zero events the honest statement is a bound: at most 14.3% at 95% confidence. Boundary rejection is weaker than containment: 18/21 prohibited conditions were rejected by the compiler, bytecode verifier, or provider registry, Wilson 95% [65.4, 95.0]%. The one that is not rejected is the declared sandboxed external operation, which is accepted and reported as *planned* without executing—by design, and confirmed by the sensors. Containment and rejection are therefore different proportions, and we report both.

C. Defects the campaign found, and the re-measurement

Three conditions failed against the preregistered oracle, and all three were product defects rather than harness artifacts.

Unchecked provider payload types. The bytecode verifier validated a tool's or model's provider and capability but never checked that the input and output types it names are declared in the program. A model bound to a non-existent type therefore passed verification and executed as a simulated dry-run call. The verifier now checks all four slots, gated on the bytecode versions whose schema carries a type table so that pre-v0.18 programs are not retroactively rejected.

No source binding. The evidence bundle referenced bytecode, trace, and report, but nothing tied the run to a source file, so editing only the source was invisible. The compiler now binds the source digest into the emitted bytecode—only the compiler can assert this, because the VM never sees the source—the bundle records the source path it was given, and offline verification recomputes the digest. A bundle that names a source the bytecode does not bind fails closed. Programs built straight from the lowering API, and multi-file packages, carry no binding and therefore make no source claim.

No producer binding. Nothing distinguished the original artifact set from a self-consistent replacement. A detached Ed25519 signature over the bundle's canonical bytes now does, and because the bundle binds every artifact by digest, one signature covers the set.

Re-running the campaign from a clean directory moved outcome accuracy from 103/106 to 118/118, evidence-modification detection from 20/22 to 21/22, and boundary rejection from 15/21 to 18/21. Nothing else changed. The pre-fix campaign, its oracle outcomes, and its raw rows are archived unedited beside the current results, and the two expectations that changed because the product changed are recorded as oracle amendments rather than applied silently.

D. What the campaign still does not show

No defects remain open. What remains are 5 boundaries, published with the same weight as the results because each one bounds a claim, and expanded in Section efsec:limits: unsigned verification cannot detect a self-consistent replacement, signing adds no key governance, the mediation-point observations describe the evaluation build, the declared sandboxed operation is not rejected at a boundary, and the injection result is containment rather than resistance.

The last is the one most easily overread. One model, one temperature and seed, eight scenarios; repetitions are not independent tasks and are not counted as such. Making resistance a claim Argorix could support needs content-bounded messages and a runtime-selected dispatch instruction, both future work.

VI. Security Interpretation and Limitations

The evaluated architecture places a decision point before provider invocation, and the campaign now measures it from outside the runtime rather than reading the runtime's own verdict. A control's presence is still not the same as its effectiveness against attacks. Four boundaries remain decisive.

Policy coverage. The historical rule vocabulary and evaluator are incompatible for all 44 rules exercised, and no governance-effectiveness claim is drawn from that snapshot. The campaign shows that the current release evaluates known rules to pass, deny, and review, and rejects an unknown rule at compilation and again at bytecode verification. It does not show coverage of a policy vocabulary the project did not author; an independent policy corpus and rule-level mutation testing remain necessary.

Prompt injection. Evaluated as containment, and only as containment. Adversarial content, a real model, utility and attack arms, and externally checked outcomes are all present, as AgentDojo and InjecAgent require [10], [11]; a prompt-bearing execution path inside Argorix is not, because the release has none. The model therefore sits outside the system under test and a published, total mapping carries its proposal in. One model on eight scenarios supports no statement about models in general, and nothing here supports "Argorix prevents prompt injection".

Side effects. Network, secret, and filesystem sensors are independent of the VM, each demonstrated a positive control from outside, and a separate evaluation build demonstrated one from inside a provider invocation. 0/21 sink hits is therefore a measurement rather than an absence of instrumentation. The claim it supports is still bounded. It covers the release process under this harness, not the operating system; the mediation-point observations describe the evaluation binary rather than the release; and no real external adapter exists to test. One of the eight conditions is not rejected by any boundary—the declared sandboxed operation, by design—so "blocked before the destination" remains a claim about the sink, not about mediation.

Evidence authenticity. A fixed bundle detects 21/22 of 22 independent post-generation modifications to its referenced artifacts, source edits now included, and a trust anchor rejects 3/3 non-producer sets. The separation still matters: integrity says the artifacts agree with each other, authenticity says who produced them, and only the second needs a key. What we added is a signature, not key governance—no storage, rotation, revocation, or trusted timestamping—and an unsigned bundle continues to make no authenticity claim at all. An append-only or transparency service remains necessary for provenance over time [7]–[9].

Other limitations follow from scope. The workloads, oracle, and sensors are authored by the same project, and no third party has replicated the campaign; separating collection from scoring reduces circularity but does not remove authorship bias. Twelve workloads and 198 scored runs give wide intervals, so every proportion is published with its Wilson bound. There is no performance baseline beyond the recorded stage latencies, no live identity or credential verification, and no claim that declared jurisdiction equals legal status or physical data location. The harness, cases, oracle, raw rows, and checksums are archived so that these boundaries can be re-tested rather than inferred.

VII. Discussion

The main empirical lesson is that integrity and authorization must be reported on separate axes. A bundle can be perfectly consistent while preserving a failed or unevaluable policy decision. Systems that collapse these predicates into one "passed" flag risk turning evidence plumbing into false assurance. ArgorixLang's explicit artifacts make the contradiction visible; the camera-ready analysis treats it as a failure signal rather than a success metric.

The second lesson is methodological. The removed 57-row matrix failed not because its numbers were wrong but because the same program computed the expected and the observed value. Splitting them into two files and two processes, and adding a control that deletes a binary, converts "the harness agrees with itself" into a falsifiable statement. The cost is modest, and the separation earned its keep twice: it caught a definitional defect in our own preregistration, recorded as an amendment rather than silently repaired, and it turned every mismatch into a product fix instead of an adjusted expectation.

A third lesson concerns where mediation happens, and what it costs to watch it. We expected the runtime provider

boundary to be the interesting surface; no program the compiler accepts reaches it, because unsupported providers, unknown rules, undeclared tools, and missing capabilities are all refused earlier. That is a favourable property that also made the missing payload-type check visible, since nothing downstream would have caught it either—and it is why the release resists being measured at that point at all. Compiling a second binary behind a non-default feature resolves the tension only if the two are told apart everywhere they appear, which is why the evaluation build reports a different version and every manifest records it.

The injection result carries a last lesson, about denominators. Reporting only containment would have produced 25/25, which reads like a strong defence; reporting only attack success would have produced 20/40, which reads like a weak model. Neither alone is the finding. The pair is: the injection moved the model about half the time, and every prohibited action it proposed was contained. Separating the two also surfaced something we would otherwise have missed—the model proposed a prohibited action in 5/40 arms with no injection at all, which a design without a benign control would have silently credited to the attack.

Jurisdiction metadata follows the same discipline. Compilation can ensure a field is present, well-shaped, and propagated to evidence; it cannot make a self-assertion true. A future resolver would add trust anchors, revocation paths and availability failures, and should be evaluated on its own rather than bundled into the current language claim.

VIII. Conclusion

ArgorixLang brings agent declarations, policy metadata, provider boundaries, and evidence generation into a compiled path. An executed campaign of 231 runs over the release executables, with expected and observed outcomes held in separate files and separate processes, reports 118/118 typed outcomes matching a preregistered oracle, 16/16 adverse conditions ending fail-closed, 21/22 independent post-generation modifications detected relative to the original bundle, and 0/21 prohibited proposals reaching an instrumented local sink whose sensors each demonstrated a positive control. A producer trust anchor additionally rejects 3/3 non-producer artifact sets, and against a real model whose behaviour indirect injections changed in 20/40 injected arms, 25/25 prohibited proposals were contained before any sensor. The historical snapshot reproduces exactly and still does not demonstrate governance: 27 internally consistent bundles preserve 1,188 unknown-rule findings with minimal structural diversity.

The bounded claim is therefore precise. In this harness the implementation compiles and mediates the tested structures, separates declarative contracts from executable providers, keeps adverse conditions fail-closed, binds the evidence to the source it was compiled from, and preserves runtime outcomes in offline-checkable artifacts. Unsigned verification does not detect a coordinated replacement of the artifact set, signing adds no key governance, the release cannot be instrumented at its own mediation point, and no real external adapter exists to test. The injection result is containment of a proposal a model outside the system made, on one model and eight scenarios, not prompt-injection resistance. Authenticated identity, legal jurisdiction, operating-system isolation, and replacement-resistant provenance remain outside every claim made here.

Artifact Availability

Harness, cases, oracle, row-level results, checksums, and the camera-ready source are at https://github.com/argorixlabs/argorixlang; the evaluated release is archived at https://doi.org/10.5281/zenodo.21007563. The campaign reproduces from a clean directory with `python evaluation/adversarial/run.py --clean`, which runs the anti-circularity test, collection, scoring, and table generation in order; two clean runs produced identical metrics, gates, and tables for every family except the model-dependent one. Each number here is generated into `tables/` and `\input`, never transcribed. Canary values are synthetic and no real credential, endpoint, or prompt is published. The authors are affiliated with organizations developing ArgorixLang and disclose that as a potential competing interest.

References

[1] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[2] J. Ligatti and S. Reddy, "A theory of runtime enforcement, with results," in *Computer Security – ESORICS 2010*. Springer, 2010, pp. 87–100.

[3] C. Cutler, C. Eline, M. Niu, N. Rungta, C. Schlesinger, and M. Sridharan, "Cedar: A new language for expressive, fast, safe, and analyzable authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, 2024.

[4] Argorix Labs, "ArgorixLang," Software release, 2026, archived at doi:10.5281/zenodo.21007563. [Online]. Available: https://github.com/argorixlabs/argorixlang

[5] J. Pohle and T. Thiel, "Digital sovereignty," *Internet Policy Review*, vol. 9, no. 4, 2020.

[6] F. von Scherenberg, M. Hellmeier, and B. Otto, "Data sovereignty in information systems," *Electronic Markets*, vol. 34, 2024.

[7] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium*, 2019, pp. 1393–1410.

[8] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *18th USENIX Security Symposium*, 2009, pp. 317–334.

[9] B. Schneier and J. Kelsey, "Cryptographic support for secure logs on untrusted machines," in *7th USENIX Security Symposium*, 1998.

[10] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[11] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.

[12] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.

[13] R. Raskar, P. Chari, J. Zinky *et al.*, "Beyond DNS: Unlocking the internet of AI agents via the NANDA index and verified AgentFacts," *arXiv preprint arXiv:2507.14263*, 2025.

[14] World Wide Web Consortium, "Decentralized identifiers (DIDs) v1.0," W3C, Recommendation, 2022. [Online]. Available: https://www.w3.org/TR/did-core/

[15] ——, "Verifiable credentials data model v2.0," W3C, Recommendation, 2025. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/